VitalFlow V1 Audit Logging Baseline

Status: Design proposal. A substantial portion of the infrastructure is already live — this document audits what exists today, identifies gaps vs. the compliance requirements, and proposes a minimal extension migration + an app-level event helper.

---

1. What exists today

Storage

- `audit.audit_events` table with columns: `id, occurred_at, tenant_id, actor_id, request_id, table_schema, table_name, row_id, action, before, after, ip, user_agent, impersonator_id`.
- `audit.log_change()` `SECURITY DEFINER` trigger function:
  - Captures `INSERT / UPDATE / DELETE` row diffs as `before` / `after` jsonb.
  - Uses `auth.uid()` for `actor_id`.
  - Pulls `impersonator_id` from `public.current_impersonation()`.
  - Does **not** yet populate `request_id`, `ip`, `user_agent` (requires per-request config GUC).

Coverage

Triggers are attached to ~30 tables in `public`: `allergies, appointments, attachments, charges, claim_lines, claims, encounter_notes, encounters, immunizations, integration_connections, inventory_items, inventory_transactions, invitations, invoice_lines, invoices, locations, medications, order_results, orders, patient_contacts, patient_coverages, patients, payers, payments, prescriptions, problems, signatures, staff_schedules, subscriptions, tasks, tenant_members, tenants, vitals, workflow_runs`.

Gaps

- ❌ **No triggers on**: `ai_requests`, `ai_completions`, `ai_feedback`, `ai_embeddings`, `impersonation_sessions`, `platform_admins`, `feature_flags`, `feature_flag_overrides`, `patient_portal_links`, `entitlements`, `webhook_deliveries`.
- ❌ **No semantic event column** — `action` only holds `INSERT/UPDATE/DELETE`. Events like "encounter.signed" or "ai.draft_generated" are inferrable from diffs but not named.
- ❌ **No app-level event helper** — login, logout, impersonation-start, AI-draft-generation need to be logged from app code, not DB triggers.
- ❌ **No request context propagation** — `request_id`, `ip`, `user_agent` are empty because triggers can't see the HTTP request.

2. Event taxonomy (what must be logged)

Each event has a canonical **event type** — `<domain>.<action>` in snake_case. The `action` column (legacy) keeps `INSERT/UPDATE/DELETE`; the new `event_type` column (see §4) holds the semantic name.

2.1 Session & identity (app-level, not DB triggers)

Event type	Trigger	Actor	Required details
`auth.login`	Successful `signInWithPassword`	user	`method=password`, `ip`, `user_agent`
`auth.login_failed`	Failed sign-in	attempted email	`reason`, `ip`, `user_agent`
`auth.logout`	`signOut`	user	`ip`, `user_agent`
`auth.password_reset_requested`	Forgot-password form	email (if exists)	`ip`
`auth.password_changed`	Password update	user	`ip`

Note: Supabase already logs these at the GoTrue API level (see `mcp__claude_ai_Supabase__get_logs service=auth`). Mirroring into `audit.audit_events` is for unified search + in-app audit viewer. Low priority — query Supabase Auth logs directly until the mirror is built.

2.2 Tenant membership & invites (DB triggers already cover)

Event type	Underlying table	Notes
`member.invited`	`invitations` INSERT	Currently captured as `action=INSERT`. Semantic name added by app when calling the invite action.
`member.invite_accepted`	`invitations` UPDATE where `accepted_at` set	Currently captured. Needs `event_type` to distinguish from cancellation.
`member.invite_cancelled`	`invitations` UPDATE where `status='cancelled'`	Same.
`member.added`	`tenant_members` INSERT	Already captured.
`member.roles_changed`	`tenant_members` UPDATE where `roles` changed	Triggered today; needs semantic tag.
`member.removed`	`tenant_members` UPDATE where `deleted_at` set	Triggered today.

2.3 Clinical & patient (DB triggers already cover, need semantic tagging)

Event type	Source	Notes
`patient.created`	`patients` INSERT	
`patient.updated`	`patients` UPDATE	`details` should include changed-column list for masking.
`patient.merged`	`patients` UPDATE where `merged_into_id` set	App action tags.
`encounter.opened`	`encounters` INSERT	
`encounter.completed`	`encounters` UPDATE where `status='completed'`	App action tags.
`encounter.cancelled`	`encounters` UPDATE where `status='cancelled'`	
`note.created`	`encounter_notes` INSERT	
`note.updated`	`encounter_notes` UPDATE while `signed_at IS NULL`	Pre-sign edits grouped.
`note.signed`	`encounter_notes` UPDATE where `signed_at` set	App action tags; signature row also inserted.
`note.amended`	`encounter_notes` INSERT with `amended_from_id`	App action tags.
`clinical_list.item_added`	`problems` / `allergies` / `medications` INSERT	Table-specific subtypes.
`clinical_list.item_updated`	same UPDATE	
`clinical_list.item_resolved`	same UPDATE where `resolved_at` set	

2.4 AI events (mix: some DB, some app-level)

Event type	Source	Required details
`ai.draft_generated`	`ai_completions` INSERT (needs trigger)	`model`, `prompt_hash`, `patient_id` (ref), `encounter_id` (ref), `tokens_in`, `tokens_out`, `latency_ms`
`ai.draft_accepted`	`ai_feedback` INSERT where `kind='accepted'` (needs trigger)	`ai_completion_id`, `diff_ratio` (how much the human kept)
`ai.draft_rejected`	`ai_feedback` INSERT where `kind='rejected'`	`ai_completion_id`, `reason`
`ai.draft_edited_and_signed`	App action at sign time	`ai_completion_id`, `final_note_id`, `diff_ratio`

Important: PHI must NOT be in `details`. Reference IDs only. The AI prompt itself (`prompt_hash`) is fine — a salted hash, not the text.

2.5 Revenue cycle (DB triggers already cover)

Event type	Source	Notes
`charge.created/updated/voided`	`charges`	Void is UPDATE `voided_at` set.
`invoice.created/updated/issued/paid/void`	`invoices`	Status transitions become subtypes.
`claim.created/updated/submitted`	`claims`	
`claim.status_changed`	`claim_status_history` INSERT	Needs trigger added.
`claim.denied`	`claim_status_history` INSERT where `status='denied'`	Subtype of above.
`claim.appealed`	`claim_status_history` INSERT where `status='appealed'`	
`payment.recorded/refunded`	`payments`	
`write_off.applied`	`invoice_lines` UPDATE where `write_off_amount>0`	App action tags; approval-gated.

2.6 Administrative & platform

Event type	Source	Notes
`admin.setting_changed`	`tenants` UPDATE	`details` lists changed fields.
`admin.integration_connected`	`integration_connections` INSERT	
`admin.integration_disconnected`	`integration_connections` UPDATE `deleted_at` set	
`admin.feature_flag_toggled`	`feature_flags` / `feature_flag_overrides` UPDATE (needs trigger)	Platform-level.
`admin.entitlement_granted`	`entitlements` INSERT (needs trigger)	
`admin.entitlement_revoked`	`entitlements` UPDATE `revoked_at` set	

2.7 Impersonation (CRITICAL — needs both triggers and app logs)

Event type	Source	Required details
`impersonation.started`	`impersonation_sessions` INSERT (needs trigger) + app emits semantic event	`impersonator_id`, `target_user_id`, `tenant_id`, `reason`, `approved_by`, `duration_minutes`
`impersonation.ended`	`impersonation_sessions` UPDATE `revoked_at` set + app emits	`session_id`, `reason`, `ended_by` (self vs. admin)
`impersonation.action`	Every write while `current_impersonation()` non-null	Already captured: the `impersonator_id` column on each `audit_events` row.

3. Required metadata per event

Minimum fields every event carries (existing columns or new):

Field	Type	Required?	Source
`id`	uuid	✅	`gen_random_uuid()`
`occurred_at`	timestamptz	✅	`now()`
`tenant_id`	uuid	✅ for tenant-scoped events; null for platform	Row context or session
`actor_id`	uuid	✅ for authenticated events; null for system/webhook	`auth.uid()` or service user
`impersonator_id`	uuid	null unless impersonating	`current_impersonation()`
`event_type`	text	✅ (new column)	App-provided semantic name
`action`	text	✅	Trigger (`INSERT/UPDATE/DELETE`) or `APP` for app events
`table_schema` , `table_name` , `row_id`	text	✅ for row events; null for app events	Trigger context
`before` , `after`	jsonb	row diffs	Trigger
`details`	jsonb (new column)	Event-specific metadata	App or trigger
`request_id`	text	✅	Middleware via GUC `vf.request_id`
`ip`	inet	✅ where available	Middleware
`user_agent`	text	✅ where available	Middleware

4. Proposed extension migration

Add two columns + a request-context bridge:

```sql
-- Migration: 2026XXXX_audit_extensions.sql

alter table audit.audit_events
  add column event_type text,
  add column details   jsonb;

create index audit_events_event_type_idx on audit.audit_events (tenant_id, event_type, occur
create index audit_events_actor_idx on audit.audit_events (tenant_id, actor_id, occurred_at

-- Request-context capture: middleware sets these per-transaction GUCs.
-- log_change() reads them and stores into the event row.
create or replace function audit.log_change()
returns trigger language plpgsql security definer
set search_path to 'public','audit','pg_temp' as $$
declare
  v_before jsonb; v_after jsonb; v_tenant_id uuid; v_row_id text;
  v_impersonator uuid;
begin
  if tg_op = 'DELETE' then
    v_before := to_jsonb(old);
    v_tenant_id := (v_before->>'tenant_id')::uuid;
    v_row_id := v_before->>'id';
  elsif tg_op = 'UPDATE' then
    v_before := to_jsonb(old);
    v_after  := to_jsonb(new);
    v_tenant_id := coalesce((v_after->>'tenant_id')::uuid, (v_before->>'tenant_id')::uuid);
    v_row_id := v_after->>'id';
  else
    v_after := to_jsonb(new);
    v_tenant_id := (v_after->>'tenant_id')::uuid;
    v_row_id := v_after->>'id';
  end if;
  select s.impersonator_id into v_impersonator from public.current_impersonation() s;

  insert into audit.audit_events (
    tenant_id, actor_id, impersonator_id,
    table_schema, table_name, row_id,
    action, before, after,
    request_id, ip, user_agent
  ) values (
    v_tenant_id, auth.uid(), v_impersonator,
    tg_table_schema, tg_table_name, v_row_id,
    tg_op, v_before, v_after,
    nullif(current_setting('vf.request_id', true), ''),
    nullif(current_setting('vf.ip', true), '')::inet,
    nullif(current_setting('vf.user_agent', true), '')
  );
  if tg_op = 'DELETE' then return old; else return new; end if;
end; $$;

-- Attach triggers to the missing tables.
create trigger ai_completions_audit
```

```
  after insert or update or delete on public.ai_completions
  for each row execute function audit.log_change();
create trigger ai_feedback_audit
  after insert or update or delete on public.ai_feedback
  for each row execute function audit.log_change();
create trigger impersonation_sessions_audit
  after insert or update or delete on public.impersonation_sessions
  for each row execute function audit.log_change();
create trigger platform_admins_audit
  after insert or update or delete on public.platform_admins
  for each row execute function audit.log_change();
create trigger feature_flags_audit
  after insert or update or delete on public.feature_flags
  for each row execute function audit.log_change();
create trigger feature_flag_overrides_audit
  after insert or update or delete on public.feature_flag_overrides
  for each row execute function audit.log_change();
create trigger entitlements_audit
  after insert or update or delete on public.entitlements
  for each row execute function audit.log_change();
create trigger claim_status_history_audit
  after insert or update or delete on public.claim_status_history
  for each row execute function audit.log_change();
```

---

5. App-level event helper

For events that can't come from a DB trigger (login, AI draft generation context, impersonation reason), app code calls `logEvent()` from @vitalflow/auth/audit. The helper uses the **service-role client** because `audit.audit_events` has strict RLS (only `audit:read`).

Example:

```
import { logEvent } from "@vitalflow/auth/audit";

await logEvent({
  tenantId: session.tenantId,
  actorId: session.userId,
  eventType: "ai.draft_generated",
  action: "APP",
  details: {
    completion_id: completion.id,
    model: "claude-opus-4-7",
    patient_id: encounter.patient_id,
    encounter_id: encounter.id,
    tokens_in: 1842,
    tokens_out: 517,
    latency_ms: 1320,
  },
  request: { requestId, ip, userAgent },
});
```

---

6. Request-context propagation

Middleware (apps/web/src/middleware.ts) sets transaction-scoped GUCs before forwarding the request to the RSC / Server Action runtime:

```
// In a server utility called from middleware or at the top of each action:
await supabase.rpc("set_request_context", {
  p_request_id: requestId,
  p_ip: ip,
  p_user_agent: userAgent,
});
```

The RPC (new, added in the extension migration):

```
create or replace function public.set_request_context(
  p_request_id text, p_ip text, p_user_agent text
) returns void language sql as $$
  select set_config('vf.request_id', coalesce(p_request_id, ''), true),
         set_config('vf.ip', coalesce(p_ip, ''), true),
         set_config('vf.user_agent', coalesce(p_user_agent, ''), true);
$$;
```

`set_config(..., is_local=true)` scopes to the current transaction, so values don't leak across requests on the same connection.

---

7. Retention

- **Default retention**: 10 years for PHI-touching events (HIPAA § 164.316(b)(2)(i)). Non-PHI (login, feature-flag toggles) can be shorter — 2 years suffices.
- **Partitioning** (deferred): partition `audit.audit_events` by `occurred_at` monthly once the table exceeds ~50M rows. Use `pg_partman` or manual ranged partitions.
- **Archive to cold storage**: export partitions older than 2 years to S3 (Parquet), retain DB copy for 1 additional year for fast lookups.
- **Deletion**: only super_admin can truncate; always via partitioned archive + explicit `audit.purge(before_date, reason)` RPC that itself writes an event. No naked `DELETE`.

---

8. Read / search / export

Read path

- RLS on `audit.audit_events`:
  - `audit_logs:view` permission → can read rows where `tenant_id = current_tenant_id()`.
  - Super admins (via impersonation): can read any tenant's rows.
  - Patients: can read only events where they're the actor (`self:read` branch) — i.e. their own logins.

Search

- In-app admin viewer at `/admin/audit`:
  - Filter by date range, actor, event_type, table_name, row_id.
  - Full-text search over `details` → add a GIN index on `(tenant_id, details jsonb_path_ops)`.
- Indexes (from §4): `(tenant_id, event_type, occurred_at desc)` + `(tenant_id, actor_id, occurred_at desc)`.

Export

- `audit_logs:export` permission (distinct — see docs/permissions-matrix.md).
- CSV with columns: `occurred_at, event_type, action, actor_email, impersonator_email, table_name, row_id, ip, request_id, details_json`.

- Export itself emits `admin.audit_exported` event with row count + date range.
- Rate-limited: one export per tenant per 5 minutes.

---

9. Masking of sensitive data

The `before` / `after` jsonb columns contain full row snapshots, which may include PHI (patient name, DOB, address) and secrets (integration tokens).

Strategy:

1. **Integration secrets** — never stored raw. `integration_connections.secret_encrypted` is already encrypted; `before` / `after` will contain the ciphertext only, which is fine.
2. **PHI** — stored as-is; protected by:
   - Strict RLS (`audit_logs:view` permission, practice_owner / office_admin only).
   - UI-layer masking: the `/admin/audit` viewer displays `[name]` , `[dob]` , `[ssn]` placeholders unless user has `audit_logs:view_phi` (a subset perm, not in V1 matrix — TBD).
   - Export: CSV redacts by default; unredacted export requires `audit_logs:export_phi` + double approval (not V1).
3. **Auth tokens** in `user_agent` / `ip` — `ip` is required for forensics; keep as-is. `user_agent` is a fingerprint, not a secret.
4. **Psychotherapy notes** — `encounter_notes.psych=true` rows: their `before` / `after` payload is masked to `[REDACTED_PSYCH_NOTE]` at trigger time. Add a conditional in `log_change()` once the `psych` column is introduced.
5. **Column-level redaction** (deferred): per-table redaction lists, enforced in trigger. Not V1.

---

10. Middleware / hooks strategy

Layer	Responsibility
Next.js middleware	Attach `x-request-id` header; pass to server actions/RSCs via async-local-storage.
Supabase Server Client (per-request)	Call `public.set_request_context()` before any query that might log (cheap no-op if called repeatedly in same tx).
DB trigger `audit.log_change()`	Reads `vf.request_id` / `vf.ip` / `vf.user_agent` GUCs; logs the row change.
App event helper `logEvent()`	Explicit calls for non-row events (login, AI, impersonation-start). Uses service role + same request context.
Never trust	Client-provided `actor_id` or `request_id` . Always derive from session / middleware.

Failure mode: if `log_change()` raises an error, the triggering write also fails (they're in the same transaction). This is intentional for financial + clinical writes — if we can't audit it, we don't do it. App-level `logEvent` calls use **fire-and-forget** within the same tx so that a hypothetical audit-table outage doesn't block login. Track as a `logger.error` so it's observable.

---

11. Edge cases & security risks

Risk	Mitigation
Forged actor — someone modifies `auth.uid()` via a crafted JWT	Supabase signs JWTs; `log_change()` uses `auth.uid()` so forgery = system compromise already
Trigger drops during migration	CI check: fail if any `public.*` table touching PHI lacks an `_audit` trigger
App-level events bypassed	Mandatory wrapper: all server actions route through `withAudit(eventType, fn)` decorator (future)
Impersonator erases their trail	`audit.audit_events` has no DELETE permission even for super_admin (enforce via RLS + `REVOKE DELETE`)
PHI in `details` jsonb	Review gate: `details` schema per event type is validated by Zod before insert
Ever-growing table	Retention job + partitioning (§7)
Auth logs forked from Supabase	Mirror Supabase Auth events to `audit_events` via webhook (phase 2)
Log injection via user input	All `details` values are jsonb — typed, not concatenated strings
Clock skew	`occurred_at` uses server `now()` , not client clock

---

12. Compliance alignment

- **HIPAA § 164.312(b)** — audit controls: ✅ all PHI reads/writes logged; actor identified; immutable (no DELETE).
- **45 CFR 164.316(b)(2)** — 6-year retention minimum; we target 10.
- **HITECH breach definition** — unauthorized PHI access requires evidence trail → `audit_events` is the evidence table.
- **SOC 2 CC7.2** — system monitoring: admin setting changes, integration toggles, role grants all logged.
- **State-specific** (e.g., Texas HB 300): patient access to their own audit trail — exposed via `/my/security` (patient surface, deferred).

---

13. Rollout plan

Phase 0 (done): row-level trigger coverage on the 30+ clinical/billing/admin tables.

Phase 1 (this doc):

- Add `event_type`, `details` columns + indexes (see §4 migration).
- Add triggers on the 8 missing tables.
- Add `set_request_context` RPC + wire middleware.
- Ship `packages/auth/src/audit.ts` with `logEvent()` and typed event schemas.

Phase 2:

- Build `/admin/audit` viewer page (filter + search + CSV export).
- Add Supabase Auth webhook → mirror login events into `audit_events`.
- CI check: every PHI table has an audit trigger.

Phase 3:

- Partitioning + archive job.
- Patient-facing `/my/security` audit view.
- Column-level redaction config.